UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Ingeniería en Sistemas de Información

Arquitectura de Computadores II – Sección 040

Catedrático: [Nombre del catedrático]

Anexo A: Architecture Decision Records (ADR) del sistema Centinela Urbano

Documento anexo al informe final del proyecto

Integrantes del grupo [número]

[Nombre completo del integrante 1]	Carné 0000-00-00000
[Nombre completo del integrante 2]	Carné 0000-00-00000
[Nombre completo del integrante 3]	Carné 0000-00-00000
[Nombre completo del integrante 4]	Carné 0000-00-00000
[Nombre completo del integrante 5]	Carné 0000-00-00000

Guatemala, 31 de octubre de 2026

Índice

Presentación....1
ADR-001: Plataforma de despliegue on-premise (LAN municipal)....2
Contexto....2
Opciones consideradas....3
Estimación de costo (orden de magnitud, en US$, a validar con cotizaciones)….3
Latencia extremo a extremo esperada (on-premise)....4
Decisión....5
Consecuencias....5
Impacto en el negocio....6
ADR-002: Broker de eventos Apache Kafka (modo KRaft)....7
Contexto....7
Opciones consideradas....7
Decisión....8
Consecuencias....9
Impacto en el negocio....10
ADR-003: Persistencia de eventos en TimescaleDB (PostgreSQL + series de tiempo) 11
Contexto....11
Dimensionamiento....11
Opciones consideradas....12
Decisión....12
Consecuencias....13
Impacto en el negocio....14
ADR-004: Tareas periódicas con Spring Scheduler + ShedLock (en lugar de Apache Airflow)....15
Contexto....15
Opciones consideradas....16
Decisión....16
Consecuencias....17
Impacto en el negocio....17

ADR-005: Contenerización con Docker y orquestación con Docker Compose por nodo (sin Kubernetes)....19
Contexto....19
Opciones consideradas....19
Decisión....20
Consecuencias....22
Impacto en el negocio....22
ADR-006: Resiliencia y tolerancia a fallos....23
Contexto....23
Opciones consideradas....24
Decisión....25
Política de pérdida aceptable....25
Mecanismos implementados....25
Pruebas realizadas en la simulación....29
Consecuencias....30
Brechas encontradas por las pruebas y su corrección....31
Impacto en el negocio....32
ADR-007: Seguridad y privacidad de datos personales....33
Contexto....33
Opciones consideradas....34
Decisión....34
Consecuencias....36
Impacto en el negocio....37
ADR-008: Observabilidad y objetivos de nivel de servicio (SLO)....38
Contexto....38
Opciones consideradas....38
Decisión....39
SLO definidos....39
Qué ocurre cuando se incumplen....40
Resultados medidos....41
Consecuencias....41

Impacto en el negocio.....42
ADR-009: Modelo de correlación por evidencia ponderada con ventanas en Redis.....43
Contexto.....43
Opciones consideradas.....44
Decisión.....44
Consecuencias.....46
Impacto en el negocio.....47
ADR-010: Microservicios orientados a eventos en Java 21 + Spring Boot (simulador incluido).....48
Contexto.....48
Opciones consideradas.....48
Decisión.....49
Consecuencias.....50
Impacto en el negocio.....51

Presentación

Registro de las decisiones de arquitectura de **Centinela Urbano**. Cada ADR sigue el formato obligatorio de la sección 4.1 del enunciado: **contexto, opciones consideradas (mínimo dos, con sus trade-offs), decisión, consecuencias e impacto en el negocio** (redactado para un directivo no técnico).

Tabla A1

Índice de decisiones de arquitectura

#	Decisión	Tema exigido (sección 4.2)
ADR-001	Despliegue on-premise en la LAN municipal, con costo estimado y latencia extremo a extremo	Plataforma de despliegue
ADR-002	Apache Kafka (KRaft), un topic por sensor y 12 particiones	Broker de eventos
ADR-003	TimescaleDB como única base de datos; se omite Elasticsearch	Persistencia de eventos
ADR-004	Spring Scheduler + ShedLock en lugar de Airflow	Orquestación de tareas periódicas
ADR-005	Docker Compose por nodo; decisión consciente de no usar Kubernetes	Contenerización y orquestación
ADR-006	Escritura anticipada en disco, reintentos según la clase de falla, DLQ con reproceso, vigilante del nodo B y política de pérdida aceptable	Resiliencia y tolerancia a fallos
ADR-007	Seudonimización HMAC, retención de 30 días, mínimo privilegio y segmentación	Seguridad y privacidad de datos
ADR-008	4 SLO medibles y degradación controlada automática	Observabilidad y SLOs
ADR-009	Motor de evidencia ponderada con ventanas en Redis	(adicional) Modelo de correlación
ADR-010	Microservicios orientados a eventos en Java 21 / Spring Boot	(adicional) Estilo arquitectónico y stack

El enunciado exige al menos seis ADRs; aquí hay diez. Las ocho decisiones obligatorias tienen cada una su propio ADR, y se suman dos más que justifican el modelo de inferencia y el stack.

ADR-001: Plataforma de despliegue on-premise (LAN municipal)

Tabla A2

ADR-001: ficha de la decisión

Campo	Valor
Estado	Aceptado
Fecha	2026-10-09
Decisores	Equipo de arquitectura del proyecto
Relacionados	ADR-002, ADR-003, ADR-005, ADR-007

Contexto

El enunciado exige elegir y defender un modelo de despliegue para un tipo de cliente concreto. Para este proyecto se definió el siguiente **cliente hipotético**:

- **Municipalidad de tamaño medio** del área metropolitana de Guatemala: unos 500 000 habitantes, 12 zonas y un centro de monitoreo municipal que opera 24/7 junto a la Policía Municipal.
- **Presupuesto de inversión limitado** y una partida de funcionamiento anual difícil de aumentar. Comprometer un gasto mensual en dólares que varía con el uso es un riesgo financiero y administrativo.
- **Sin personal especializado en nube.** Sí cuenta con un área de informática que administra servidores Linux.
- **Datos personales sensibles:** las placas vehiculares, la ubicación de ciudadanos y los reportes identifican a personas. El área legal exige que estos datos no salgan de la infraestructura municipal (ver ADR-007).
- **Continuidad operativa:** la detección debe funcionar aunque falle el enlace a internet. El enunciado pide "evitar depender de latencias externas no controladas para la parte de ingesta".
- **Volumen moderado:** unos 13,5 eventos por segundo en operación normal, con picos de 10 a 20 veces en conciertos, partidos o feriados (unos 270 eventos/s). Opciones del enunciado: on-premise/LAN, híbrido (LAN + nube) y cloud-native.

Opciones consideradas

Tabla A3

ADR-001: opciones consideradas

Criterio	On-premise / LAN	Híbrido (ingesta local + analítica en nube)	Cloud-native
Costo inicial	Alto (hardware ≈ US$12 000)	Medio (≈ US$7 000 de hardware local)	Bajo (casi nulo)
Costo recurrente	Bajo y predecible (≈ US$7 500/año)	Medio y variable (nube + salida de datos)	Alto y variable (≈ US$11 000–14 000/año + enlace dedicado)
Control de los datos personales	Total: nunca salen de la red municipal	Parcial: el histórico se va a la nube	Bajo: todo en un proveedor externo
Latencia sensor → alerta	Muy baja: todo en LAN (medido: p50 ≈ 50 ms)	Baja para alertas (locales), alta para histórico	Depende de internet (+60 a 120 ms por tramo hacia la región más cercana)
Funciona sin internet	Sí	Las alertas sí, la analítica no	No
Elasticidad ante picos	No: se dimensiona para el pico	Parcial	Sí, automática
Complejidad operativa	Media: hay que administrar servidores	Alta: dos entornos y su integración	Media: servicios gestionados, pero hace falta experiencia en nube
Dependencia del proveedor (lock-in)	Ninguna (todo es software libre)	Media	Alta (MSK, RDS, ElastiCache…)

Estimación de costo (orden de magnitud, en US$, a validar con cotizaciones)

On-premise (opción elegida):

Tabla A4

ADR-001: estimación de costo (orden de magnitud, en US$, a validar con cotizaciones)

Concepto	Detalle	Costo
Nodo A (datos)	Servidor de 8 núcleos, 32 GB de RAM, 2 × 1 TB SSD en RAID 1	3 500
Nodo B (servicios y visualización)	Servidor de 8 núcleos, 32 GB de RAM	2 500
Nodos C/D (borde / pasarela de sensores)	2 mini PC	1 200
Red	Switch administrable con VLAN y firewall/UTM pequeño	1 500
Energía	UPS de 3 kVA y rack	1 800

Concepto	Detalle	Costo
Contingencia	15 %	1 575
Inversión inicial (CAPEX)		≈ 12 000 (≈ Q93 000)
Energía	≈ 800 W promedio, unos 7 000 kWh/año	1 750/año
Mantenimiento y garantías	≈ 10 % del CAPEX	1 200/año
Administración	0,25 de un puesto de técnico	4 500/año
Licencias	Todo el software es libre	0
Costo anual (OPEX)		≈ 7 500/año
Costo total a 3 años (TCO)		≈ 34 500

Cloud-native (referencia con precios de lista aproximados de un proveedor público):

- Kafka gestionado con 3 brokers: ≈ US$460/mes.
- PostgreSQL gestionado con alta disponibilidad: ≈ US$240/mes. TimescaleDB gestionado cuesta más.
- Redis gestionado: ≈ US$50/mes.
- Cómputo para 4 servicios: ≈ US$150/mes.
- VPN sitio a sitio y almacenamiento: ≈ US$100/mes.

Esto da ≈ **US$1 000/mes, unos US$12 000/año**. A eso hay que sumar un enlace de internet dedicado con SLA, imprescindible porque todos los sensores dependerían de él: ≈ US$4 000–7 000/año. **TCO a 3 años ≈ US$48 000–57 000**, sin inversión inicial.

Híbrido: ≈ US$7 000 de hardware local más ≈ US$400/mes de nube para histórico y analítica, más la salida de datos. **TCO a 3 años ≈ US$36 000–40 000**, con la complejidad de dos entornos.

Latencia extremo a extremo esperada (on-premise)

Tabla A5

ADR-001: latencia extremo a extremo esperada (on-premise)

Tramo	Estimado	Medido en la simulación
Sensor → Kafka (LAN)	< 5 ms	—
Ingesta (validar, normalizar,	5–20 ms	p95 de procesamiento < 10 ms

Tramo	Estimado	Medido en la simulación
seudonimizar)		
Correlación (consultas a Redis en memoria)	5–30 ms	—
Persistencia y aviso a la consola	10–50 ms	—
Total evento → alerta guardada	≈ 50–150 ms	Operación normal: p50 ≈ 45–80 ms. Pico de diseño (270 eventos/s, multi-nodo): p50 = 39 ms, p95 = 67 ms, máximo = 80 ms

El objetivo de negocio (alerta en menos de 2 s, ADR-008) se cumple con mucho margen.

Decisión

Se adopta un despliegue **on-premise sobre la LAN municipal**, repartido en los nodos de la sección 6 del enunciado:

- **Nodo A:** Kafka, TimescaleDB y Redis.
- **Nodo B:** microservicios, Prometheus y Grafana.
- **Nodos C/D:** generación de eventos (en producción serían las pasarelas de sensores) e instancias adicionales de procesamiento para repartir la carga.

Se diseña para el pico (×20) y no para el promedio. Como evolución futura queda la opción **híbrida**: enviar a la nube solo agregados anónimos (conteos por zona y hora) para analítica regional. Así se aprovecha la nube sin sacar datos personales.

Consecuencias

Positivas

- Los datos personales nunca salen de la red municipal, lo que facilita la defensa legal (ADR-007).
- La detección funciona aunque se caiga internet.
- El costo recurrente es predecible y bajo, sin facturas variables en dólares.
- Latencia mínima: en el pico de diseño se midió p95 = 67 ms, unas 30 veces por debajo del objetivo de 2 s.
- No hay dependencia de un proveedor de nube; todo el stack es software libre.

Negativas (aceptadas)

- **No hay elasticidad.** Si un evento extraordinario supera ×20 el tráfico normal, el sistema se degrada. Se mitiga con:
    - degradación controlada (ADR-008);
    - escalado horizontal agregando instancias en los nodos C/D (probado en la simulación: Kafka reparte las 12 particiones entre máquinas automáticamente).
- **Inversión inicial y responsabilidad de operar el hardware:** energía, respaldos y reemplazo de discos.
- La alta disponibilidad real requiere duplicar el nodo A (3 brokers de Kafka y una réplica de PostgreSQL). En la simulación se acepta un solo nodo A (ver ADR-006).

Impacto en el negocio

- **Costo:** una inversión única de unos Q93 000 y unos Q58 000 al año de funcionamiento. A 3 años sale entre un 30 % y un 40 % más barato que la nube, y el gasto es predecible para el presupuesto municipal.
- **Riesgo:** se elimina el riesgo de que los datos de placas y ubicaciones queden en manos de terceros, y el de quedar "ciego" si falla internet. A cambio, la municipalidad asume el riesgo de operar sus servidores, mitigado con UPS, discos en espejo y respaldos diarios.
- **Tiempo de implementación:** unas 6 a 8 semanas, incluida la compra de equipo. El sistema ya funciona en contenedores, así que la instalación toma horas una vez que el hardware está en sitio.

ADR-002: Broker de eventos Apache Kafka (modo KRaft)

Tabla A6

ADR-002: ficha de la decisión

Campo	Valor
Estado	Aceptado
Fecha	2026-10-09
Decisores	Equipo de arquitectura del proyecto
Relacionados	ADR-001, ADR-005, ADR-006, ADR-009

Contexto

Cinco tipos de sensores generan eventos continuamente: unos 13,5 por segundo en operación normal y hasta 270 por segundo en picos. Esos eventos los consumen, **cada uno a su propio ritmo**, tres procesos distintos:

1. La ingesta, que valida y normaliza.
2. El motor de correlación, en tiempo real.
3. El almacenamiento histórico en TimescaleDB.

Hace falta un intermediario (broker) que desacople a quien produce de quien consume, absorba los picos, no pierda eventos si un consumidor se cae, conserve el orden de los eventos de una misma zona (clave para correlacionar) y permita repartir el procesamiento entre varias máquinas. Por ADR-001, el broker debe correr **on-premise**.

Opciones consideradas

Tabla A7

ADR-002: opciones consideradas

Criterio	Apache Kafka (KRaft)	RabbitMQ	Servicio gestionado (SQS / Pub/Sub)
Modelo	Log distribuido y persistente: los mensajes quedan disponibles durante la retención	Colas: el mensaje se borra al consumirse	Colas o tópicos gestionados en la nube
Varios consumidores leyendo lo mismo	Nativo: cada grupo lleva su propio avance (offset)	Requiere un exchange *fanout* y una cola por	Pub/Sub sí; SQS requiere SNS + varias

Criterio	Apache Kafka (KRaft)	RabbitMQ	Servicio gestionado (SQS / Pub/Sub)
		consumidor	colas
Reprocesar historia	Sí (reiniciar el offset)	No: el mensaje ya se borró	Limitado
Orden	Garantizado por partición (llave = zona)	Por cola; se pierde con varios consumidores	SQS FIFO con límites de throughput
Throughput	Muy alto (cientos de miles de msg/s por broker)	Alto (decenas de miles de msg/s)	Alto, pero se paga por mensaje
Escalado horizontal de consumidores	Automático (reparto de particiones)	Consumidores compitiendo; sin orden	Automático
Operación on-premise	Media: desde la versión 3.3 KRaft elimina ZooKeeper	Fácil	No aplica: contradice ADR-001
Costo	Libre (Apache 2.0)	Libre (MPL 2.0)	Variable por mensaje y requiere internet

Decisión

Se adopta **Apache Kafka 3.9 en modo KRaft** (sin ZooKeeper), con este diseño de topics:

Tabla A8

ADR-002: decisión

Topic	Particiones	Retención	Llave	Uso
`sensor.panic.raw`, `sensor.lpr.raw`, `sensor.speed.raw`, `sensor.acoustic.raw`, `sensor.citizen.raw`	12	24 h	id del sensor	Eventos crudos, un topic por tipo de sensor
`events.normalized`	12	72 h	id de zona	Eventos validados y anonimizados
`alerts`	3	7 días	id de zona	Alertas del motor
`events.dlq`	1	7 días	—	Cola de mensajes muertos (ADR-006)

Razones clave de diseño:

- **Un topic por tipo de sensor:** una avalancha de lecturas LPR no retrasa los botones de pánico, porque cada topic tiene sus propios hilos consumidores. Además, cada esquema evoluciona por separado.
- **Llave = zona en `events.normalized`:** todos los eventos de una zona llegan **en orden** a la misma instancia del motor de correlación.
- **12 particiones:** permiten hasta 12 consumidores en paralelo por grupo, por ejemplo 2 máquinas con 6 hilos cada una. Se probó con el motor repartido entre los nodos B y C: Kafka asignó 6 particiones a cada uno y, al apagar el nodo C, el B tomó las 12 en segundos.
- **Productor con `acks=all` e idempotencia:** el broker confirma solo lo que quedó escrito y los reintentos no duplican mensajes.

Consecuencias

Positivas

- Tres grupos de consumidores (ingestion, correlation, event-store) leen los mismos eventos sin estorbarse. Si la base de datos se pone lenta, la correlación en tiempo real no se entera.
- Si un consumidor cae, los eventos esperan en Kafka (72 h) y se procesan al volver, sin pérdida.
- Se puede escalar agregando instancias, sin cambiar código ni configuración.
- Los mensajes son JSON legible: se pueden inyectar a mano con `kafka-console-producer` durante la demo (por ejemplo, para mostrar la DLQ).

Negativas (aceptadas)

- Kafka es más complejo de operar que RabbitMQ: particiones, grupos y retención.
- En la simulación hay un solo broker (factor de replicación 1). Si el nodo A cae, no se pueden publicar eventos: los críticos quedan en un buffer en disco (ADR-006). En producción se recomiendan 3 brokers con replicación 3.
- Consume más memoria que RabbitMQ (≈ 1 GB de heap).

Impacto en el negocio

- **Costo:** cero en licencias; corre en el servidor del nodo A ya presupuestado (ADR-001).
- **Riesgo:** reduce el riesgo de perder eventos ante caídas parciales. Kafka es el estándar de la industria para este tipo de problema, así que es fácil conseguir personal y soporte.
- **Tiempo:** la configuración ya está automatizada en Docker Compose. Crecer (más sensores, más zonas) no exige rediseñar nada: basta con agregar máquinas.

ADR-003: Persistencia de eventos en TimescaleDB (PostgreSQL + series de tiempo)

Tabla A9

ADR-003: ficha de la decisión

Campo	Valor
Estado	Aceptado
Fecha	2026-10-09
Decisores	Equipo de arquitectura del proyecto
Relacionados	ADR-001, ADR-007, ADR-008, ADR-010

Contexto

El sistema guarda dos tipos de datos muy distintos:

1. **Eventos de sensores:** gran volumen (≈ 1,2 millones diarios en operación normal), escritura continua, casi nunca se modifican y se consultan **por rangos de tiempo** ("eventos de la Zona 4 en los últimos 15 minutos", "eventos por minuto").
2. **Alertas, auditoría, lista de placas y SLO:** volumen bajo, con relaciones y actualizaciones (un operador valida una alerta). Requieren integridad transaccional.

El stack de referencia propone PostgreSQL para lo estructurado y Elasticsearch para el histórico. Grafana debe consultar ambos y existe un requisito de retención: los datos se borran o anonimizan pasado cierto tiempo (ADR-007).

Dimensionamiento

- **Operación normal:** 13,5 eventos/s, unos 1,17 millones de filas diarias.
- **Tamaño:** ≈ 400 bytes por fila más índices, unos 0,8 GB diarios sin comprimir.
- **Con 90 días de retención y compresión a partir del día 45:** ≈ 36 GB sin comprimir más ≈ 4 GB comprimidos. Cabe con holgura en el disco de 1 TB del nodo A.

Opciones consideradas

Tabla A10

ADR-003: opciones consideradas

Criterio	PostgreSQL puro	TimescaleDB (extensión de PostgreSQL)	InfluxDB	MongoDB / Elasticsearch (NoSQL)
Consultas por rango de tiempo	Buenas con índices; se degradan con cientos de millones de filas	Excelentes: particiona por tiempo automáticamente (*hypertables*)	Excelentes	Buenas (Elastic) / regulares (Mongo)
Datos relacionales y transacciones (alertas, auditoría)	Sí	Sí: es PostgreSQL	No (sin JOIN ni transacciones)	Limitadas
Compresión	No nativa	Nativa por columnas (≈ 90 % menos)	Nativa	Parcial
Retención automática	Manual	Política nativa (`add_retention_policy`)	Nativa	Manual (Elastic ILM)
Agregados precalculados	Vistas materializadas manuales	Agregados continuos que se actualizan solos	Tareas	Agregaciones al consultar
Lenguaje	SQL	SQL estándar más funciones de tiempo	Flux/InfluxQL (otro lenguaje)	Propio
Integración con Grafana	Nativa	Nativa (modo TimescaleDB)	Nativa	Nativa
Bases que operar	1	1	2 (Influx + PostgreSQL para alertas)	2
Recursos	Bajos	Bajos	Bajos	Elastic ≈ 2 GB de RAM mínimo

Decisión

Se adopta **TimescaleDB 2.17 sobre PostgreSQL 16** como **única** base de datos:

- **`sensor_events` es una hypertable** con particiones (*chunks*) diarias, índices por zona, tipo y placa seudonimizada, compresión a partir de los 45 días y borrado automático a los 90 días.

- **Agregado continuo `events_per_minute`**, con agregación en tiempo real: Grafana grafica millones de eventos sin recorrerlos uno por uno.
- **Tablas relacionales normales** para `alerts`, `alert_revisions`, `audit_log`, `plate_watchlist`, `slo_measurements` y `rule_quality`. Su volumen es bajo y necesitan actualizaciones y llaves primarias.
- **Vistas `v_events` y `v_alerts`** sin seudónimos, para el usuario de solo lectura de Grafana (ADR-007).
- **Esquema versionado con Flyway** desde el código Java (`alert-service/src/main/resources/db/migration`).

Se descarta Elasticsearch. La búsqueda histórica que se necesita (por zona, tipo, placa seudonimizada y rango de fechas) se resuelve con índices de TimescaleDB. Elasticsearch duplicaría los datos, consumiría unos 2 GB más de RAM y sumaría una tercera base que operar, a cambio de búsqueda de texto libre que hoy solo aplicaría al campo "mensaje" de los reportes ciudadanos. Si en el futuro se necesita, se puede alimentar desde Kafka con un nuevo grupo de consumidores sin tocar nada de lo existente.

Consecuencias

Positivas

- **Una sola base de datos:** un solo respaldo, un solo motor que aprender, un solo usuario de Grafana. Los eventos y las alertas se pueden cruzar con SQL.
- La retención y la compresión son políticas declarativas: no hay que programar tareas de borrado de eventos.
- Las inserciones son por lotes e idempotentes (`ON CONFLICT DO NOTHING`): si Kafka reentrega un lote, no se duplican filas.
- Medido en la simulación: en el pico de 270 eventos/s, los datos aparecen en Grafana en 47 ms (p95 de frescura).

Negativas (aceptadas)

- Las funciones avanzadas (compresión y agregados continuos) usan la licencia Timescale License (TSL). Es gratuita para uso propio, pero no permite ofrecer TimescaleDB como servicio a terceros. Esto no afecta a una municipalidad.

- Los agregados continuos no se pueden crear dentro de una transacción: esa migración de Flyway se marca como no transaccional (`V2__agregados_continuos.sql.conf`).
- No hay búsqueda de texto libre avanzada (ver arriba).

Impacto en el negocio

- **Costo:** sin licencias y sin un servidor adicional para Elasticsearch, lo que ahorra unos 2 GB de RAM y su administración. Con la compresión, el disco del nodo A alcanza para años.
- **Riesgo:** PostgreSQL es una base madura de más de 25 años, con amplia disponibilidad de personal en Guatemala. Con el borrado automático de datos viejos, se cumple la política de retención sin depender de que alguien lo recuerde.
- **Tiempo:** el esquema se crea solo al arrancar el sistema, sin pasos manuales.

ADR-004: Tareas periódicas con Spring Scheduler + ShedLock (en lugar de Apache Airflow)

Tabla A11

ADR-004: ficha de la decisión

Campo	Valor
Estado	Aceptado
Fecha	2026-10-09
Decisores	Equipo de arquitectura del proyecto
Relacionados	ADR-003, ADR-007, ADR-008, ADR-009, ADR-010

Contexto

El stack de referencia asigna a Apache Airflow las "tareas periódicas de análisis, validación y entrenamiento". El enunciado pregunta explícitamente: **¿se justifica la complejidad operativa de Airflow a esta escala?** Las tareas que necesita el sistema son:

Tabla A12

ADR-004: contexto

Tarea	Frecuencia	Qué hace
Monitor de SLO	Cada 30 s	Mide la latencia y la frescura de datos y activa la degradación controlada (ADR-008)
Retención de placas	Diaria (02:15)	Borra seudónimos de placas de más de 30 días (ADR-007)
Expiración de alertas	Cada 5 min	Pasa a EXPIRED las alertas que nadie atendió en 2 h
Calidad de reglas	Cada hora	Calcula la precisión por regla y zona y ajusta los umbrales del motor (ADR-009)
Sincronización de la lista de placas	Cada 10 min	Copia los seudónimos de PostgreSQL a Redis
Revisión de la lista de placas	Cada 15 s	Si Redis se reinició vacío, recarga la lista de inmediato (ADR-006)
Reproceso de la DLQ	Cada 5 min	Reinyecta lo que falló por causas no permanentes, hasta 3 veces (ADR-006)

Son **7 tareas independientes, cortas (de milisegundos a segundos) y sin dependencias entre sí**. No hay flujos de varios pasos, reintentos encadenados ni reprocesos de fechas pasadas (*backfill*). Restricción adicional: el curso exige programar en Java, y Airflow solo acepta flujos (DAGs) en Python.

Opciones consideradas

Tabla A13

ADR-004: opciones consideradas

Criterio	Apache Airflow	Quartz Scheduler en clúster	Spring `@Scheduled` + ShedLock
Para qué está pensado	Flujos de datos complejos con dependencias (DAGs)	Planificación avanzada (calendarios, persistencia de disparos)	Tareas periódicas simples dentro de la aplicación
Componentes extra	Webserver, scheduler, workers, base de metadatos y, opcionalmente, Redis/Celery	Tablas de Quartz en la base de datos	Una tabla (`shedlock`)
RAM adicional	≈ 2–4 GB	≈ 0 (dentro de la JVM)	≈ 0 (dentro de la JVM)
Lenguaje de las tareas	Python (choca con "Java obligatorio")	Java	Java
Interfaz para ver ejecuciones	Excelente	No	No (se usa la auditoría, los logs y Grafana)
Evitar ejecuciones duplicadas con varias instancias	Sí	Sí (clúster)	Sí (bloqueo distribuido en la base de datos)
Curva de aprendizaje	Alta	Media	Baja (anotaciones)

Decisión

Se usa **Spring `@Scheduled` con ShedLock (proveedor JDBC sobre TimescaleDB)** dentro del `alert-service`.

- Cada tarea se declara con `@Scheduled` (cron o intervalo) y `@SchedulerLock(name=...)`.
- **ShedLock garantiza que cada tarea corra en una sola instancia**, aunque haya varias copias del `alert-service` en distintas máquinas. Usa la hora de la base de

datos (`usingDbTime()`), así que no le afectan relojes desincronizados entre nodos.

- Para la demo, las tareas se pueden disparar manualmente con `POST /api/admin/jobs/{tarea}` (rol ADMIN).
- Las acciones que tocan datos personales (purga de placas) quedan en la bitácora `audit_log`.

Cuándo sí convendría Airflow: si el proyecto creciera hacia el entrenamiento real de modelos de aprendizaje automático, con flujos de varios pasos (extraer → limpiar → entrenar → validar → publicar), reprocesos de meses anteriores o decenas de tareas con dependencias. Esa revisión queda registrada como el disparador para reconsiderar esta decisión.

Consecuencias

Positivas

- No hay que operar 4 a 5 contenedores más ni reservar 2 a 4 GB de RAM.
- Todo el código del sistema queda en Java y se prueba con las mismas herramientas.
- La tarea de calidad de reglas cumple el rol de "análisis y entrenamiento" del enunciado: aprende de las validaciones de los operadores y ajusta los umbrales del motor.

Negativas (aceptadas)

- No hay una interfaz gráfica de ejecuciones como la de Airflow. Se compensa con logs, la bitácora de auditoría y las tablas `slo_measurements` y `rule_quality` visibles en Grafana.
- Si el `alert-service` está caído, sus tareas no corren. Con varias instancias, cualquiera de ellas toma el bloqueo.

Impacto en el negocio

- **Costo:** se evita el equivalente a un servidor pequeño (2 a 4 GB de RAM) y la capacitación en Airflow.

- **Riesgo:** menos piezas significa menos puntos de falla. El riesgo de no tener Airflow es bajo, porque las tareas son simples, y la decisión indica cuándo revisarla.
- **Tiempo:** se implementó en horas en lugar de días; agregar una tarea nueva es escribir un método con una anotación.

ADR-005: Contenerización con Docker y orquestación con Docker Compose por nodo (sin Kubernetes)

Tabla A14

ADR-005: ficha de la decisión

Campo	Valor
Estado	Aceptado
Fecha	2026-10-09
Decisores	Equipo de arquitectura del proyecto
Relacionados	ADR-001, ADR-002, ADR-006

Contexto

El sistema tiene 9 componentes: 4 microservicios Java, Kafka, TimescaleDB, Redis, Prometheus y Grafana. Se reparten entre 3 o 4 máquinas del laboratorio. La tabla de nodos del enunciado *recomienda* Ubuntu para el nodo A, macOS/Linux para el B y Windows/Linux para C/D, pero las computadoras disponibles del grupo son Windows 10/11, y el requisito del equipo es poder levantar todo desde Windows sin usar Linux. Para la puesta en simulación hay que levantarlos de forma repetible, con la misma configuración en todos los equipos, y el grupo debe poder repartir la carga entre varias máquinas. El enunciado pide defender tanto la adopción de Kubernetes como la **decisión consciente de no usarlo**.

Opciones consideradas

Tabla A15

ADR-005: opciones consideradas

Criterio	Instalación directa (sin contenedores)	Docker Compose por nodo	Kubernetes (k3s / kubeadm)
Portabilidad Windows/macOS/Linux	Baja: cada SO se instala distinto	Alta: misma imagen en todos	Alta
Tiempo de puesta en marcha	Horas por máquina	Minutos (`docker compose up`)	Horas: montar el clúster, la red (CNI) y el almacenamiento

Criterio	Instalación directa (sin contenedores)	Docker Compose por nodo	Kubernetes (k3s / kubeadm)
Escalado	Manual	Manual: se agregan instancias en otro nodo (*profile* `workers`)	Automático (HPA)
Recuperación ante fallas	Manual	`restart: unless-stopped` en el mismo host	Reprograma los pods en otro nodo
Curva de aprendizaje	Baja	Baja	Alta (pods, services, ingress, PVC, RBAC)
Consumo extra	Ninguno	Mínimo	Plano de control: ≈ 1–2 GB de RAM por nodo de control
Red entre máquinas	Configuración manual	Puertos publicados en la LAN	Red superpuesta (*overlay*); delicada en redes de laboratorio
Adecuado para 3–4 máquinas y 9 componentes	No	Sí	Sobredimensionado

Decisión

Se usa **Docker** para empaquetar cada componente y **Docker Compose** para orquestarlos, con dos modos:

1. **Una sola máquina** (`docker-compose.yml` en la raíz): todo el sistema con un comando. Sirve para desarrollo y como plan B en la demo.
2. **Multi-nodo** (`deploy/nodo-a`, `deploy/nodo-b`, `deploy/nodo-c`): un archivo Compose por rol. Los nodos se comunican por IP en la LAN; la variable `NODE_A_IP` indica dónde están Kafka, TimescaleDB y Redis.

   Detalles:

- **Un único `Dockerfile` multi-etapa** para los 4 servicios (`--build-arg MODULE=...`):
  - `runtime` compila con Maven dentro de Docker, así que solo hace falta Docker instalado;
  - `prebuilt` usa los `.jar` ya compilados y no necesita internet en el laboratorio.
- **Imágenes sin privilegios:** usuario `centinela`, no root.

- **Repartir la carga:** `docker compose --profile workers up -d` en los nodos C/D levanta instancias extra de ingesta y correlación; Kafka les reparte las 12 particiones. Probado: 6 particiones por nodo y reasignación automática al apagar uno.
- **Llevar las imágenes al laboratorio:** se construyen antes y se transportan con `docker save` / `docker load` (`centinela.cmd exportar-imagenes` / `importar-imagenes`).
- **Operación desde Windows:** `centinela.cmd` (PowerShell 5.1/7, sin bash ni WSL a mano) configura cada PC (detecta la IP, reparte zonas, genera la llave HMAC), revisa los puertos que Windows reserva para Hyper-V/WSL y elige otros libres, abre el firewall con una sola elevación, levanta los nodos en orden esperando a que estén sanos y muestra el estado de toda la red. Los puertos publicados son variables de `.env`, iguales en todas las PCs.
- **SO de cada nodo:** como todo corre en contenedores, el mismo archivo Compose funciona en Windows (Docker Desktop), Linux o macOS. Se usa Windows 10/11 en todas las PCs; el SO del enunciado es recomendado, no obligatorio, y la elección no cambia la arquitectura. Docker Desktop administra internamente su propia máquina virtual: el equipo no instala ni opera Linux.

¿Por qué no Kubernetes?

- Su principal valor, reprogramar contenedores automáticamente y escalar según la carga, rinde a partir de decenas de nodos o servicios.
- Con 3 o 4 máquinas, su plano de control consumiría una parte importante de la RAM disponible.
- Su red superpuesta añade puntos de falla justo el día de la demo en vivo.
- El escalado que este sistema necesita (agregar consumidores) ya lo resuelve Kafka, sin un orquestador.

Cuándo migrar a Kubernetes:

- más de unas 10 máquinas;
- requisitos de alta disponibilidad con reprogramación automática entre nodos;
- despliegues frecuentes sin cortes de servicio (*rolling updates*);
- o un modelo híbrido o en la nube (ADR-001, evolución futura).

Las imágenes ya son compatibles con Kubernetes, así que la migración no exige cambios de código.

Consecuencias

Positivas

- Cualquier integrante del grupo levanta el sistema completo en su laptop con un comando.
- La misma imagen corre en desarrollo, en el laboratorio y en "producción".
- La configuración queda como código, versionada en Git.

Negativas (aceptadas)

- No hay reprogramación automática entre máquinas: si un nodo se apaga, sus contenedores no se mueven solos a otro. En la práctica se mitiga porque Kafka reasigna el trabajo de los consumidores a los que siguen vivos (probado).
- Todas las PCs deben compartir el mismo `.env` (IP, puertos y secretos). El script lo genera en la primera PC y detecta el rol de las demás por su IP, pero hay que copiar el archivo a mano.
- En Windows, Docker Desktop tiene particularidades que el script cubre: puertos reservados por Hyper-V/WSL (error 10013), firewall de entrada, memoria de la VM y reloj de la VM tras suspender.

Impacto en el negocio

- **Costo:** sin licencias ni hardware extra para un plano de control. Se aprovechan las máquinas que ya existen.
- **Riesgo:** baja el riesgo operativo de la demo y del día a día, porque hay menos piezas y es más fácil diagnosticar. El riesgo de no tener alta disponibilidad automática se acepta para la escala actual y tiene definido su disparador de revisión.
- **Tiempo:** de horas a minutos para poner en marcha cada máquina. El personal municipal no necesita formación en Kubernetes.

ADR-006: Resiliencia y tolerancia a fallos

Tabla A16

ADR-006: ficha de la decisión

Campo	Valor
Estado	Aceptado (revisado el 2026-10-15 tras las pruebas de caída de nodos)
Fecha	2026-10-09
Decisores	Equipo de arquitectura del proyecto
Relacionados	ADR-002, ADR-003, ADR-005, ADR-008, ADR-009

Contexto

En un sistema de seguridad ciudadana, no todos los eventos valen lo mismo. Perder la pulsación de un botón de pánico puede costar una vida; perder una de las miles de lecturas de velocidad normales de un minuto no cambia nada. El enunciado pide definir:

- el manejo de caídas del broker;
- la cola de mensajes muertos (DLQ);
- los reintentos;
- la **pérdida aceptable de eventos**.

La arquitectura tiene dos servidores con papeles distintos: el **nodo A** guarda el estado (Kafka, TimescaleDB, Redis) y el **nodo B** lo procesa (ingesta, correlación, alertas, Grafana). Los nodos C/D generan los eventos y pueden correr *workers* de ingesta y correlación.

Fallas que hay que contemplar:

1. Kafka no disponible: cae el nodo A o la red.
2. Mensajes mal formados: un sensor defectuoso o un ataque.
3. Caídas temporales de Redis o de la base de datos.
4. Caída del nodo B o de una instancia de un microservicio.
5. Muerte de un proceso a mitad de una operación (alerta publicada pero estado sin guardar).
6. Duplicados generados por los reintentos.

La primera versión de este ADR se probó apagando de verdad cada nodo. Las pruebas encontraron ocho brechas entre lo que el ADR prometía y lo que el sistema hacía; esta revisión las corrige y deja constancia de ellas en la sección final.

Opciones consideradas

Tabla A17

ADR-006: opciones consideradas

Aspecto	Opción A	Opción B	Elegida
Mensajes inválidos	Descartarlos y registrarlos en el log	Enviarlos a una DLQ con la causa	B: se pueden revisar y reprocesar
Reintentos	Un número fijo para todo error	Según la clase de falla (transitoria, permanente, desconocida)	B: una caída de Redis de 5 minutos no debe mandar alertas a la DLQ
Kafka caído, en el productor	Guardar en disco solo lo que ya falló	**Escritura anticipada (*write-ahead*)*: a disco antes de enviar	B: no hay ventana en que un crítico viva solo en memoria
Qué guardar si cae Kafka	Solo los críticos	Críticos sin vencimiento (tope 32 MB) + telemetría útil con tope de 15 min	B: sin la telemetría, un robo durante la caída se clasificaba como falso positivo
Garantía de entrega	Como máximo una vez (*at-most-once*)	Al menos una vez + deduplicación + IDs deterministas	B
Estado de la correlación	En la memoria de cada proceso	En Redis compartido	B: cualquier instancia puede reemplazar a otra
Detección de la caída de B	Que alguien note que la consola no carga	Vigilante en el nodo C + aviso en la consola	B
Alta disponibilidad del broker	1 broker	3 brokers con replicación 3	1 en la simulación; 3 recomendado en producción

Decisión

Política de pérdida aceptable

Tabla A18

ADR-006: política de pérdida aceptable

Tipo de evento	Pérdida aceptable	Mecanismo
Botón de pánico, reporte ciudadano, disparo o explosión con probabilidad ≥ 0,5 (simulador)	0	Spool *write-ahead* con `fsync`, sin vencimiento; si el disco se llena (32 MB ≈ 100 000 eventos) se rechaza lo nuevo y se registra, nunca se descarta lo guardado
Pánico y reporte desde la app móvil (gateway `/api/v1/panic`)	0	Mismo spool en la ingesta: 202 `EN_COLA` si Kafka no responde; 503 con `Retry-After` solo si tampoco hay disco (la app reintenta con el mismo `event_id`)
Telemetría útil durante una caída de Kafka: todas las placas (LPR), excesos de velocidad, contravía y sonidos relevantes	Se conserva la de los últimos 15 min (tope 32 MB)	Spool de telemetría: vence a los 15 min porque más vieja ya no corrobora nada (ventanas de 15 min); si se llena, se descarta la más antigua
Lecturas de velocidad normales y ruido ambiental durante una caída de Kafka	Se pierden	No participan en ninguna regla; guardarlas llenaría el disco sin beneficio
Lecturas de velocidad normales en modo degradado (sobrecarga real)	Se conserva 1 de cada 10	Muestreo en la ingesta (ADR-008)
Cualquier evento ya aceptado por Kafka	0 (mientras la caída del consumidor dure menos que la retención)	Retención de 24 h en los topics crudos y 72 h en `events.normalized`, offset confirmado solo después de procesar y reintentos sin límite para fallas transitorias
Memoria de corto plazo (ventanas, incidentes abiertos) si se reinicia Redis	Se pierde	Aceptado: Redis corre sin persistencia por rendimiento. La lista de placas robadas, que sí importa, se reconstruye sola desde la base

Mecanismos implementados

1. Productores confiables. `acks=all` e `enable.idempotence=true`. El broker confirma solo lo escrito y los reintentos internos no duplican. `max.block.ms` limita a 3–5 s

la espera de `send()` sin broker, para que un nodo A caído no congele al simulador ni al gateway.

*2. *Spool* write-ahead* en disco **(`common/.../spool/DiskSpool`). Cada evento crítico se escribe en un diario en disco** antes** de enviarlo y se marca como confirmado cuando Kafka responde:

- formato de líneas con suma de verificación CRC32C (`P` = pendiente, `A` = confirmado, `F` = falla), de modo que una línea a medio escribir por un corte de luz se detecta y se ignora;
- `fsync` agrupado (varios eventos por escritura física) y compactación atómica (archivo nuevo + `rename`) cuando el diario crece;
- al arrancar, se relee el diario y se reenvía lo pendiente: un `kill -9` del simulador con Kafka caído no pierde nada;
- al volver Kafka se drena primero lo crítico y luego la telemetría, a 500 eventos/s para no saturar al nodo B;
- los eventos reenviados llevan las cabeceras `centinela-first-attempt` y `centinela-spooled`. La ingesta los marca (`reenviado_spool`, `retraso_origen_s`) y mide su antigüedad contra la hora original, de modo que no se rechazan por "viejos" tras una caída larga.

Tabla A19

ADR-006: mecanismos implementados

Spool	Qué guarda	Tope	Al llenarse
`critical.wal` (simulador)	Pánico, reporte ciudadano, disparo/explosión ≥ 0,5	32 MB, sin vencimiento	Rechaza lo nuevo y lo registra en el log; se ve como `rechazadosPorTope` en `/api/simulator/spool` y el evento perdido cuenta en `simulator_events_sent_total{result="lost"}`
`telemetry.wal` (simulador)	LPR, excesos, contravía, sonidos relevantes	32 MB o 15 min	Descarta lo más antiguo
`gateway.wal` (ingesta)	Pánicos y reportes de la app	32 MB, sin vencimiento	Responde 503 para que la app reintente

3. Reintentos según la clase de falla (`FailureClassifier` + `ResilientErrorHandlers`, iguales en los tres consumidores):

Tabla A20

ADR-006: mecanismos implementados

Clase	Ejemplos	Reintentos	Destino final
PERMANENTE	JSON inválido, campos faltantes, zona inexistente, timestamp en el futuro	Ninguno	DLQ de inmediato
TRANSITORIO	Kafka, Redis o la base no responden (`SQLState 08*/57P*`, conexión rechazada, *timeout*)	Sin límite, con espera exponencial de 1 s a 30 s	Ninguno: el mensaje espera en Kafka hasta que vuelva la dependencia
DESCONOCIDO	Cualquier otro error	4	DLQ

Mientras reintenta, el consumidor no avanza el offset, así que el mensaje no se pierde ni se salta. Los reintentos se registran cada 10 intentos para no inundar el log.

4. DLQ con reproceso. Cada mensaje en `events.dlq` lleva el topic, la partición y el offset originales, la causa y tres cabeceras propias: `centinela-error-class`, `centinela-service` y `centinela-dlq-attempts`. El alert-service corre cada 5 min (con ShedLock, una sola instancia) una tarea que **reinyecta en su topic original** lo que falló por causas transitorias o desconocidas, hasta 3 veces por mensaje. Lo permanente solo se reprocesa a pedido de un administrador (`centinela.cmd reprocesar-dlq -IncluirPermanentes`, auditado), por ejemplo tras corregir una regla de validación. `centinela.cmd dlq` muestra el total por clase y los últimos mensajes.

5. Deduplicación con ficha de entrega. La ingesta reclama cada `event_id` en Redis con un script Lua atómico que guarda también la "ficha" de la entrega (`topic:partición:offset`):

- `CLAIM`: si el `event_id` no existe, lo marca como *pendiente* (60 s) con la ficha; si existe con otra ficha, es un duplicado real y se descarta;
- `CONFIRM`: al publicar el evento normalizado, la marca pasa a *confirmada* (10 min);
- `RELEASE`: si el procesamiento falla, se borra la marca para que el reintento no se tome por duplicado.

Así, un reintento del **mismo** mensaje no se descarta (antes podía perderse) y un duplicado real (otro envío del mismo evento) sí. Si el proceso muere con la marca

pendiente, vence sola a los 60 s. En la base de datos se mantiene `ON CONFLICT DO NOTHING` y en las alertas el *upsert* por `alert_id` y revisión.

6. Interruptor de Redis. Si Redis cae, la ingesta **sigue publicando sin deduplicar** (se prefiere disponibilidad: los duplicados se neutralizan en la base y en las alertas) y un único hilo vuelve a probar Redis cada 5 s, en lugar de que cada evento espere el *timeout* de 2 s. El motor de correlación, que sí necesita Redis para su estado, trata la caída como falla transitoria: reintenta sin límite y nada va a la DLQ. Resultado: un solo aviso en el log y recuperación automática.

7. Motor de correlación atómico. El motor **publica la alerta antes de guardar su estado** (marcar evidencia como usada, abrir el incidente, registrar al sospechoso) y cada alerta tiene un **ID determinista** (UUID derivado de la regla, la zona y el evento ancla). Si el proceso muere entre la publicación y el guardado, Kafka le entrega el evento a otra instancia, que recalcula la misma alerta con el mismo ID, y la persistencia idempotente la deja una sola vez. Nunca queda estado guardado de una alerta que no salió.

8. Protección contra falsos positivos en la puesta al día. Si el motor procesa eventos con más de 10 s de atraso (Kafka acumuló mensajes durante una caída), lo anota en Redis (`correlation:backlog`, 20 s) y **ninguna instancia clasifica falsos positivos** mientras tanto: la evidencia que corrobora un pánico puede venir detrás en la cola. Si la publicación de un falso positivo falla, el ancla vuelve a la lista de pendientes.

9. Lista de placas robadas autorreparable. La sincronización desde la base es atómica (se arma en una llave temporal y se renombra) y deja una llave centinela `watchlist:synced`. El alert-service revisa la centinela cada 15 s: si falta (Redis se reinició vacío), recarga la lista y los ajustes de umbral de inmediato. El motor conserva su última copia buena mientras tanto, en lugar de quedarse con una lista vacía.

10. Servicios sin estado local y rebalanceo rápido. Las ventanas, los incidentes y los sospechosos viven en Redis. Si una instancia cae, Kafka reasigna sus particiones a otra en ~10 s (`session.timeout.ms` = 10 s, latido cada 3 s; el valor por defecto era 45 s).

11. Vigilante del nodo B (en el simulador del nodo C). Cada 10 s consulta la salud de los servicios de B (con 3 s de espera y 2 fallas seguidas para declarar caído, para no dar falsas alarmas) y, con la API de administración de Kafka, qué grupos de

consumidores tienen miembros y cuánto atraso acumulan. Reporta **OPERATIVO / DEGRADADO / SIN_RESPUESTA**, distingue "la máquina no responde" de "el servicio no responde" y dice **dónde sigue la detección** (p. ej. "La detección continúa en: correlation-engine-nodo-c") y qué se está acumulando sin consumidor. Se ve en `/vigilante.html`, en `centinela.cmd estado` y en el log.

12. Consola consciente de la conexión. Si se corta el canal en vivo (SSE), la consola muestra "Sin conexión con el servidor desde las HH:MM… reintentando". Al reconectar, pide las alertas generadas durante el corte, las **destaca** y marca cuánto tardaron ("LLEGÓ 91 s TARDE"), para que el operador sepa que no las vio en su momento.

13. SLO que distingue sobrecarga de recuperación (ADR-008). Si el SLO se incumple, se mira el atraso de los cuatro grupos de consumidores: si es grande y no baja, es **SOBRECARGA** (se activa la degradación); si es bajo o va bajando, es **RECUPERACIÓN** de una caída ya superada: se avisa una vez al operador (alerta MEDIUM "Puesta al día tras una caída") y **no** se degrada, porque descartaría datos justo durante la puesta al día.

14. Operación. Reinicio automático de contenedores (`restart: unless-stopped`); apagado ordenado (el simulador y la ingesta terminan la ronda de reenvío y cierran el diario antes de salir); `/actuator/health/liveness` separado de la salud completa, para que Docker no reinicie un servicio sano solo porque Redis está caído; respaldo de la base con `pg_dump` cada 6 h (se guardan 8; una base vacía no se respalda y un reinicio no rota respaldos buenos) y restauración con `centinela.cmd restaurar`, que restaura en una base temporal y luego la intercambia por la actual, sin detener los servicios.

Pruebas realizadas en la simulación

Todas en modo multi-nodo (A, B y C con *workers*, como proyectos Docker independientes conectados por IP), con tráfico de fondo y la consola del operador abierta en un navegador automatizado.

Tabla A21

ADR-006: pruebas realizadas en la simulación

Prueba	Resultado
Caída del nodo B (90 s) con un escenario de disparos	El vigilante declaró NODO B SIN RESPUESTA a los 16 s e informó "La detección continúa en: correlation-engine-nodo-c". El worker de C generó DISPAROS_CONFIRMADOS 10 s después del escenario, con B caído. La consola mostró "Sin conexión…"; al volver B: "Conexión restablecida tras 107 s: 1 alerta…", destacada como "LLEGÓ 91 s TARDE". El SLO registró RECUPERACIÓN, sin modo degradado. 0 eventos perdidos.
Caída del nodo A (82 s) con un robo en curso y un pánico desde la app	El gateway respondió 202 `EN_COLA`; el simulador guardó 4 críticos y 222 eventos de telemetría útil. Al volver A el spool se vació en 3 s, la lista de placas se recargó sola a los 20 s y el robo salió como ROBO_EN_CURSO HIGH → CRITICAL (antes: falso positivo). El pánico de la app generó su alerta. El vigilante dijo "Causa probable: el nodo A (datos) no responde". SLO: RECUPERACIÓN. 0 críticos perdidos.
Redis caído 45 s con un escenario de disparos	0 mensajes a la DLQ; un solo aviso por servicio y ninguna traza de error; la alerta salió al volver Redis.
`kill -9` del simulador con Kafka caído	Al arrancar, el diario devolvió el evento pendiente, se reenvió y salió su alerta. 0 perdidos.
3 mensajes inválidos inyectados con `kafka-console-producer`	Los 3 llegaron a `events.dlq` como PERMANENTE, sin reintentos; el reproceso por defecto no los tocó. El resto del flujo siguió.
Worker de correlación del nodo C apagado	Kafka reasignó sus 6 particiones al nodo B; una alerta de disparos se generó en 188 ms con el worker caído. Al volver, 6 y 6.
Pruebas automatizadas del diario, la clasificación de fallas, la deduplicación, el gateway, el vigilante, el SLO y el motor	95 pruebas, todas en verde (incluye: línea truncada por corte de luz, compactación, reinicio con pendientes, reintento del mismo mensaje, muerte entre publicar y guardar).

Consecuencias

Positivas

- Ningún evento crítico depende de la memoria de un proceso: está en disco antes de salir.
- Una caída de B solo produce retraso, y ahora se **ve**: el vigilante la anuncia y la consola avisa.

- Una caída de A ya no convierte un robo real en falso positivo, ni deja la lista de placas vacía.
- Una caída larga de Redis o de la base ya no manda alertas a la DLQ; lo que llega a la DLQ se reprocesa solo si la causa era pasajera.
- Un mensaje dañado no bloquea el sistema ("mensaje veneno") y queda disponible para análisis.

Negativas (aceptadas)

- Con un solo broker y una sola base, el nodo A sigue siendo el **punto único de falla**: mientras está caído no se detecta nada nuevo, solo se guarda para después.
- "Al menos una vez" implica posibles duplicados, que la deduplicación y los IDs deterministas neutralizan (con un pequeño costo en Redis).
- El spool vive en la máquina generadora: si esa máquina pierde su disco durante una caída de Kafka, esos eventos se pierden.
- La memoria de corto plazo de Redis se pierde si se reinicia: un incidente abierto antes de la caída de A se abre como nuevo si llega más evidencia.
- `fsync` por grupo de eventos críticos cuesta unos milisegundos por evento; es irrelevante para su volumen (decenas por minuto).

Brechas encontradas por las pruebas y su corrección

Tabla A22

ADR-006: brechas encontradas por las pruebas y su corrección

#	Brecha	Corrección (mecanismo)
1	Al volver A, la lista de placas robadas quedaba vacía hasta 10 min	Centinela revisada cada 15 s y sincronización atómica (9)
2	Un robo durante la caída de A se clasificaba como falso positivo	Spool de telemetría útil (2) y pausa de clasificación en la puesta al día (8)
3	Nadie avisaba que B había caído	Vigilante en C (11) y aviso en la consola (12)
4	Al recuperarse, el SLO activaba el modo degradado sin sobrecarga	Causa del incumplimiento por atraso de Kafka (13)
5	El spool se escribía cuando el envío ya había fallado: hasta 30 s en memoria	Escritura anticipada con `fsync` (2)
6	El gateway de la app respondía 500 si caía A	Spool en el gateway, 202 `EN_COLA` (2)
7	Lo que agotaba los reintentos iba a la DLQ y	Reintentos por clase de falla (3) y reproceso de la

#	Brecha	Corrección (mecanismo)
	nadie lo reprocesaba	DLQ (4)
8	No había respaldo de la base	`pg_dump` programado y restauración (14); en producción, réplica de la base y Kafka con factor de réplica 3

Impacto en el negocio

- **Costo:** sin costo adicional en la simulación. La alta disponibilidad completa en producción (3 brokers, una réplica de la base y Redis con Sentinel) costaría unos US$4 000 más en hardware.
- **Riesgo:** queda explícito qué se puede perder y qué no, una definición que el área legal y la dirección pueden revisar y aprobar. La pulsación de un ciudadano en peligro no se pierde aunque falle el servidor central, y si cae el servidor de servicios el supervisor se entera en segundos.
- **Tiempo:** los mecanismos están incluidos y probados; la recuperación no requiere intervención manual.

ADR-007: Seguridad y privacidad de datos personales

Tabla A23

ADR-007: ficha de la decisión

Campo	Valor
Estado	Aceptado
Fecha	2026-10-09
Decisores	Equipo de arquitectura del proyecto (con revisión del área legal)
Relacionados	ADR-001, ADR-003, ADR-004

Contexto

El sistema procesa datos que **identifican o permiten rastrear a personas**:

- **Placas vehiculares:** junto con la ubicación y la hora de cada cámara permiten reconstruir los desplazamientos de una persona.
- **Ubicación de ciudadanos** que presionan el botón de pánico o envían reportes desde la app; puede revelar su domicilio.
- **Identificadores de usuarios y dispositivos** de la app.
- **Texto libre** en los reportes ciudadanos.

El enunciado exige definir cómo se protegen, anonimizan y retienen, **aunque los datos sean sintéticos**.

En Guatemala, el Decreto 57-2008 (Ley de Acceso a la Información Pública) protege los datos personales sensibles en poder de entidades públicas y regula el habeas data. Una municipalidad es sujeto obligado de esa ley. Como referencia internacional se adoptan los principios del Reglamento General de Protección de Datos europeo (RGPD): minimización, limitación de la finalidad, limitación del plazo de conservación y privacidad desde el diseño. También se toma como referencia comparada la práctica policial británica para lectores de placas (ANPR), que fija plazos máximos de conservación de las lecturas.

Opciones consideradas

Tabla A24

ADR-007: opciones consideradas

Técnica para placas	Ventaja	Desventaja	Decisión
Guardar la placa en claro, con control de acceso	Investigación sencilla	Una fuga expone los movimientos de miles de personas	Descartada
Cifrado reversible (AES)	Recuperable por orden judicial	Requiere gestión de llaves; sigue siendo dato personal pleno	Descartada para el almacenamiento general
Hash simple (SHA-256)	Irreversible a simple vista	Reversible por fuerza bruta: las placas guatemaltecas tienen pocas combinaciones ($\approx 10^7$)	Descartada
Seudonimización HMAC-SHA256 con llave secreta + placa enmascarada	Sin la llave es irreversible; la misma placa da el mismo seudónimo, lo que permite correlacionar y seguir vehículos	Si la llave se filtra, hay que rotarla	Elegida

Tabla A25

ADR-007: opciones consideradas

Control de acceso	Decisión
API y consola sin autenticación	Descartada
Autenticación con roles (OPERATOR, ADMIN) y bitácora de auditoría	Elegida (HTTP Basic en la simulación; en producción, OIDC/LDAP municipal)

Decisión

1. **Seudonimización en la puerta de entrada.** El `ingestion-service` reemplaza la placa por `HMAC-SHA256(llave, placa)` (128 bits) y una versión enmascarada para el operador (`XYZ***`). **La placa en claro no se escribe nunca** en Kafka normalizado, Redis, la base de datos ni los logs.

    - La lista de vehículos robados también se guarda seudonimizada: el administrador ingresa la placa, se seudonimiza al instante y se compara por seudónimo.
    - Los identificadores de ciudadanos y de dispositivos de la app también se seudonimizan.
2. **Generalización de ubicación:** las coordenadas que provienen de ciudadanos se redondean a 3 decimales (≈ 110 m). Alcanza para despachar una patrulla y evita señalar una vivienda exacta.
3. **Minimización:**
    - del contexto del usuario de la app solo se conservan la plataforma y la versión, no el id;
    - el mensaje libre se recorta a 200 caracteres.
4. **Retención limitada:**

Tabla A26

ADR-007: decisión

Dato	Plazo	Cómo se aplica
Eventos crudos en Kafka	24 h	Retención del topic
Eventos normalizados en Kafka	72 h	Retención del topic
Seudónimo de placa en eventos y alertas no confirmadas	30 días	Tarea diaria `PlateRetentionJob`, auditada
Eventos (sin seudónimo)	90 días	Política de retención de TimescaleDB
Alertas confirmadas	Se conservan (posible proceso legal)	Se excluyen de la purga
Estado de correlación en Redis	Minutos a 1 h	Expiración (TTL) de cada llave

1. **Mínimo privilegio:**
    - Grafana usa el usuario `grafana_reader`, que solo lee vistas **sin seudónimos** (`v_events`, `v_alerts`);
    - los operadores ven alertas y validan, pero no modifican la lista de placas;
    - solo ADMIN gestiona la lista y ejecuta tareas administrativas.

2. **Auditoría:** cada validación de alerta, cambio en la lista de placas, purga de datos y reinicio del estado de correlación queda en `audit_log` con quién, qué y cuándo.
3. **Secretos fuera del código:** la llave HMAC y las contraseñas van en variables de entorno (`.env`, excluido de Git). Si se usa la llave de desarrollo, el servicio emite una advertencia al arrancar.
4. **Segmentación de red** (esquema de infraestructura):

Tabla A27

ADR-007: decisión

Segmento	Equipos	Puede alcanzar
VLAN 10, sensores/borde	Nodos C/D, pasarelas	Solo el nodo A: Kafka 9092 y Redis 6379
VLAN 20, servicios	Nodo B	Nodo A: 9092, 5432 y 6379
VLAN 30, datos	Nodo A	No inicia conexiones
VLAN 40, operadores	Consolas del centro de monitoreo	Solo el nodo B: 8083 y 3000

Contenedores sin privilegios (usuario `centinela`) y Redis con contraseña.

Pendiente para producción, aceptado en la simulación: TLS en Kafka, PostgreSQL y HTTP; SASL/SCRAM con ACL por servicio en Kafka; y la llave HMAC en un gestor de secretos con rotación anual.

Consecuencias

Positivas

- Una fuga de la base de datos **no revela placas ni identidades**, solo seudónimos inútiles sin la llave.
- El sistema sigue correlacionando, siguiendo vehículos en fuga y comparando con la lista de robados sin conocer ninguna placa.
- Los plazos de conservación se cumplen de forma automática y verificable.
- Grafana, el componente más expuesto visualmente, no tiene acceso a seudónimos.

Negativas (aceptadas)

- Para identificar una placa ante un juez hace falta un procedimiento formal: aplicar el HMAC a la placa sospechosa para buscarla, o consultar los registros de la cámara de origen. La búsqueda inversa no es posible por diseño.
- Rotar la llave HMAC rompe la continuidad de seudónimos entre periodos; se planifica anual.
- La generalización a 110 m reduce algo la precisión del despacho.

Impacto en el negocio

- **Costo:** prácticamente nulo; son decisiones de diseño implementadas en el software.
- **Riesgo:** reduce de forma drástica el riesgo legal y reputacional de una fuga de datos. Es el argumento central ante el área legal, que la municipalidad puede presentar a la ciudadanía como "vigilancia con privacidad por diseño".
- **Tiempo:** ya implementado. El paso a producción requiere unos 2 a 3 días para certificados TLS y la integración con el directorio de usuarios municipal.

ADR-008: Observabilidad y objetivos de nivel de servicio (SLO)

Tabla A28

ADR-008: ficha de la decisión

Campo	Valor
Estado	Aceptado
Fecha	2026-10-09
Decisores	Equipo de arquitectura del proyecto
Relacionados	ADR-003, ADR-004, ADR-006, ADR-009

Contexto

El enunciado pide definir **umbrales concretos**, por ejemplo una alerta generada en menos de 2 segundos desde el evento, y **qué ocurre cuando se incumplen, más allá de tener dashboards en Grafana**. Un tablero en rojo que nadie mira no protege a nadie: el sistema debe medir, decidir y actuar.

Hay dos tipos de información que observar:

- **Negocio:** alertas, eventos, falsos positivos, latencia de las alertas. Vive en TimescaleDB.
- **Técnica:** throughput, lag de los consumidores de Kafka, memoria, tiempos por etapa. Vive en los procesos.

Opciones consideradas

Tabla A29

ADR-008: opciones consideradas

Criterio	Solo Grafana sobre TimescaleDB	Prometheus + Grafana + acción automática	Stack completo (ELK + Jaeger + Prometheus)
Métricas de negocio	Sí	Sí (TimescaleDB)	Sí
Métricas técnicas (lag, JVM, histogramas)	No	Sí (Micrometer → Prometheus)	Sí
Trazas distribuidas	No	No: se usa el `event_id` como identificador de trazabilidad	Sí

Criterio	Solo Grafana sobre TimescaleDB	Prometheus + Grafana + acción automática	Stack completo (ELK + Jaeger + Prometheus)
Acción ante un incumplimiento	Visual	Visual + automática (degradación y alerta de sistema)	Visual + alertas
Recursos	Bajos	Bajos (Prometheus ≈ 100 MB)	Altos (≈ 4 GB o más)

Decisión

Prometheus (métricas técnicas expuestas por Micrometer en `/actuator/prometheus`) y **Grafana** con dos fuentes: TimescaleDB para el negocio y Prometheus para lo técnico. A eso se suma un **monitor de SLO en Java** que actúa ante los incumplimientos.

SLO definidos

Tabla A30

ADR-008: sLO definidos

SLO	Indicador (SLI)	Objetivo	Ventana
SLO-1 Latencia de alerta	Tiempo entre el evento que completa la correlación y la alerta guardada y visible en la consola, para avisos HIGH/CRITICAL (primer aviso y agravamientos)	95 % por debajo de 2 s	5 min móviles
SLO-2 Pérdida de eventos críticos	Botones de pánico y reportes en estado `lost`	0	Siempre
SLO-3 Frescura de datos	p95 del tiempo entre el evento y su fila guardada en TimescaleDB	< 10 s	1 min
SLO-4 Rezago de procesamiento	Atraso total de los grupos `ingestion`, `correlation`, `event-store` y `alert-store` en Kafka	< 1 000 mensajes (incumple si es ≥ 1 000 y no baja en dos mediciones seguidas)	2 mediciones de 30 s

Qué ocurre cuando se incumplen

El `SloMonitorJob` corre cada 30 s (ADR-004). Si SLO-1 o SLO-3 se incumplen, primero determina la **causa**, porque no toda alerta tardía es una sobrecarga:

Tabla A31

ADR-008: qué ocurre cuando se incumplen

Causa	Cómo se reconoce	Qué se hace
SOBRECARGA	El atraso de los cuatro grupos de consumidores de Kafka es grande (≥ 1 000 mensajes) y no baja	Los pasos 1 a 5 de abajo
RECUPERACIÓN	El atraso es bajo o va bajando: las alertas tardías son eventos que esperaron durante una caída ya superada (ADR-006)	Se registra con su causa y se avisa una vez al operador (alerta MEDIUM "Puesta al día tras una caída"). No se degrada: descartaría datos justo mientras el sistema se pone al día

Si el atraso no se puede medir, se usa el último minuto: si también incumple, se trata como sobrecarga. Ante una SOBRECARGA:

1. **Degradación controlada automática:** activa la bandera `system:degraded` en Redis. La ingesta lo detecta en menos de 5 s y empieza a **conservar solo el 10 % de las lecturas de velocidad normales**, el mayor volumen del sistema, que no participa en la correlación. Así se libera capacidad para pánicos, disparos y lecturas anómalas, que nunca se descartan.
2. **Alerta de sistema `SLO_INCUMPLIDO`** en la consola del operador, con el detalle (p95, porcentaje de cumplimiento, frescura): el supervisor sabe que los tiempos no son confiables.
3. **Registro** en `slo_measurements`, con la causa y el atraso de Kafka, para el informe de cumplimiento (panel "Incumplimientos de SLO").
4. **Procedimiento (runbook) para el supervisor:**
   - revisar el panel de lag por servicio;
   - si el cuello de botella es la correlación o la ingesta, levantar *workers* en un nodo C/D (`WORKERS=true` en `.env` y `centinela.cmd iniciar`), y Kafka reparte la carga solo;

- si es la base de datos, la correlación en tiempo real sigue funcionando y lo histórico se pone al día al recuperarse.

5. **Recuperación:** cuando la medición vuelve a cumplir, se borra la bandera y la ingesta vuelve a guardar todo.

Resultados medidos

Tabla A32

ADR-008: resultados medidos

Escenario	Volumen	p50	p95	Máx.	¿Cumple SLO-1?
Operación normal (validación de 9 escenarios)	13,5 ev/s	44–78 ms	< 500 ms	477 ms	Sí
Pico de diseño ×20 (versión final, multi-nodo)	270 ev/s	39 ms	67 ms	80 ms	Sí
Sobrecarga ×100 (primera versión, 1 nodo)	1 350 ev/s	37 s	240 s	275 s	No: se activó la degradación y la alerta de sistema
Sobrecarga ×100 (tras optimizar el motor)	1 350 ev/s	70 ms	5,0 s	8,4 s	No, pero con una mejora de 50 veces; el cuello de botella pasó a la ingesta
Caída del nodo B durante 90 s (puesta al día)	13,5 ev/s	—	90,7 s	90,7 s	No, causa RECUPERACIÓN (atraso 0): aviso MEDIUM, sin degradar. Antes de la corrección se clasificaba como sobrecarga y activaba la degradación por error

La prueba de sobrecarga se usó como **ejercicio de capacidad**. Las métricas de lag mostraron que el motor de correlación era el cuello de botella. Se aplicaron dos optimizaciones: filtrar temprano los eventos que no pueden coincidir con ninguna regla y enviar los comandos de cada ventana a Redis en un solo viaje (*pipeline*). Con ellas el lag del motor quedó en 0. Más allá de unas 5 veces el pico de diseño, la respuesta es escalar horizontalmente (ADR-005).

Consecuencias

Positivas

- Los SLO son medibles, están registrados y **se aplican solos**: el sistema protege lo crítico sin intervención humana.

- La trazabilidad funciona de punta a punta con el `event_id`: cada alerta lista los eventos de su evidencia, que pueden buscarse en `sensor_events`.
- Tres dashboards provisionados como código (centro de monitoreo, SLO y salud, falsos positivos).

Negativas (aceptadas)

- No hay trazas distribuidas de tipo Jaeger: se considera suficiente el `event_id` más las métricas por etapa.
- La medición de latencia depende de que los relojes de los nodos estén sincronizados; se exige el servicio de hora de Windows (NTP) en todas las PCs, y `centinela.cmd estado` avisa si hay desfase.
- La degradación sacrifica detalle estadístico de tránsito (velocidades normales) mientras dura.

Impacto en el negocio

- **Costo:** Prometheus usa unos 100 MB de RAM, sin licencias.
- **Riesgo:** la dirección recibe un compromiso medible ("95 % de las alertas en menos de 2 s") y evidencia de cumplimiento. Cuando hay sobrecarga, el sistema prioriza solo las emergencias en lugar de colapsar de forma pareja.
- **Tiempo:** incluido y funcionando. El supervisor solo necesita conocer el procedimiento de escalado, que se resume en un comando.

ADR-009: Modelo de correlación por evidencia ponderada con ventanas en Redis

Tabla A33

ADR-009: ficha de la decisión

Campo	Valor
Estado	Aceptado
Fecha	2026-10-09
Decisores	Equipo de arquitectura del proyecto
Relacionados	ADR-002, ADR-004, ADR-006, ADR-008

Contexto

El núcleo del proyecto es **correlacionar eventos de fuentes heterogéneas** para inferir incidentes, como en la tabla 2.4 del enunciado:

- un botón de pánico en la Zona 4 a las 22:05, más la placa XYZ123 en una cámara, más un vehículo a 85 km/h, significan un **robo en curso**;
- un botón de pánico en la Zona 10 sin ningún evento complementario es un **posible falso positivo**.

Requisitos:

- decidir en menos de 2 s (ADR-008);
- tolerar eventos que llegan desordenados;
- **explicar** por qué se generó cada alerta: el operador y un juez deben entenderla;
- ajustarse sin recompilar;
- funcionar con varias instancias en distintas máquinas;
- y contemplar falsos positivos (por ejemplo, la pirotecnia en Guatemala dispara sensores acústicos).

Opciones consideradas

Tabla A34

ADR-009: opciones consideradas

Criterio	Reglas en código (if/else)	Motor propio de evidencia ponderada, reglas en YAML, estado en Redis	Kafka Streams (uniones con ventanas)	Motor CEP (Esper / Apache Flink)	Aprendizaje automático (clasificador)
Explicabilidad	Media	Alta: cada alerta lista su evidencia y su peso	Media	Media	Baja (caja negra)
Cambiar reglas sin recompilar	No	Sí (YAML)	No	Sí (EPL/SQL)	Reentrenar
Correlacionar más de 2 fuentes con ventanas flexibles	Difícil	Sí	Complejo (uniones encadenadas)	Sí	Sí
Estado compartido entre instancias	No	Sí (Redis)	Estado local por partición	Clúster propio	—
Datos etiquetados necesarios	No	No	No	No	Sí: no existen
Infraestructura extra	No	Redis (ya en el stack)	No	Clúster de Flink	Pipeline de entrenamiento
Curva de aprendizaje	Baja	Baja	Media-alta	Alta	Alta

Decisión

Se implementa un **motor de reglas de evidencia ponderada** (`correlation-engine`):

- **Regla = ancla + evidencias.**
  - El ancla es el evento que inicia la sospecha, por ejemplo un botón de pánico de tipo ROBO.
  - Las evidencias son condiciones con peso: exceso de velocidad (+0,30), vehículo a alta velocidad en una cámara LPR (+0,20), placa en la lista de robados (+0,20), disparo o vidrios rotos (+0,15), otro testigo (+0,10).

- **Puntaje** = peso del ancla + suma de las evidencias encontradas en la **misma zona** dentro de la ventana `[ancla − antes, ancla + después]`, con un máximo de 1,0.
    - Puntaje ≥ umbral (0,70): se genera la alerta.
    - Puntaje ≥ umbral crítico (0,90): la severidad es **CRITICAL**.
- **Una persona o un sensor no puede corroborarse a sí mismo:** dentro de cada evidencia, cada fuente cuenta una sola vez.
- **Ventanas deslizantes en Redis** (*sorted sets* por zona y tipo, con la hora como puntaje). Se evalúa cada vez que llega un evento, sea ancla o evidencia, por lo que el **desorden no importa**.
- **Incidentes:**
    - las correlaciones del mismo tipo en la misma zona durante 10 min forman **un solo incidente**;
    - se emite una revisión cuando sube la severidad o aparece la placa (ESCALATED) o cuando se suma un nuevo testigo (UPDATED). No se repite la alerta.
- **Falsos positivos:**
    - un ancla que no se corrobora en 90 s produce `POSIBLE_FALSO_POSITIVO`, con severidad baja;
    - un dispositivo que presiona pánico más de 5 veces en 10 min se marca como sospechoso y se ignora durante 1 h;
    - un disparo aislado (posible pirotecnia) no basta: hacen falta 2 señales.
- **Seguimiento entre zonas:** la placa de un robo queda "sospechosa" 30 min. Si aparece en otra zona, se genera `VEHICULO_EN_FUGA`, vinculada al incidente original.
- **Umbrales adaptativos:** la tarea horaria de calidad (ADR-004) calcula la precisión por regla y zona a partir de las validaciones de los operadores.
    - Si la precisión es menor al 50 %, se exige más evidencia en esa zona (+0,10 al umbral).
    - Si se omiten incidentes reales (falsos negativos), se relaja (−0,05).
- **Rendimiento:**

- filtrado temprano: los eventos que no cumplen ninguna condición de ninguna regla (≈ 99 % de la telemetría normal) no se guardan ni se evalúan;
- los comandos de cada ventana van a Redis en un solo viaje (pipeline).

Las 8 reglas (R1 a R7 en YAML, más el seguimiento de placas y el abuso de dispositivos) están documentadas en el documento de correlación del repositorio (docs/correlacion.md), y la configuración vive en `correlation-engine/src/main/resources/application.yml`.

Por qué no aprendizaje automático: no hay datos históricos etiquetados de incidentes reales, y una caja negra es difícil de defender ante un juez o la ciudadanía. El diseño deja preparado el camino: las validaciones de los operadores (`CONFIRMED` / `FALSE_POSITIVE`) generan exactamente el conjunto etiquetado que haría falta para entrenar un modelo en una fase posterior.

Consecuencias

Positivas

- **Explicabilidad total:** "Robo en curso en Zona 4. Puntaje 0,90 (umbral 0,70). Evidencia: botón de pánico ROBO (+0,40); exceso de velocidad 85 km/h (+0,30); placa XYZ*** a 87 km/h (+0,20)".
- Reglas ajustables por un analista sin programar; 15 pruebas unitarias usan **las mismas reglas del YAML** (tabla 2.4, falsos positivos, abuso, fuga, disparos, incendio, accidente, umbral adaptativo).
- **Validado en vivo:** 9 de 9 correlaciones esperadas detectadas en cada corrida, con latencias de 28 a 477 ms.
- Escala de forma horizontal: estado en Redis y llave de zona en Kafka.

Negativas (aceptadas)

- Los pesos se fijaron por criterio experto y no se aprendieron de datos. Se mitiga con los umbrales adaptativos y la revisión periódica de la precisión (dashboard de falsos positivos).
- Puede haber falsos positivos por coincidencia, por ejemplo un pánico real junto a un exceso de velocidad no relacionado. En la simulación aparecen y los operadores los marcan, lo que alimenta el ajuste de umbrales.

- Redis es una dependencia crítica: si cae, el motor espera y reintenta hasta 2 min (ADR-006).

Impacto en el negocio

- **Costo:** sin licencias ni clúster adicional (Flink). Usa el Redis ya presupuestado.
- **Riesgo:** cada alerta se puede justificar ante la dirección, el área legal o un juez. Los falsos positivos, que gastan patrullas y desgastan la confianza ciudadana, se miden y se reducen con el uso del sistema.
- **Tiempo:** una regla nueva, por ejemplo "aglomeración anómala", se agrega en minutos editando la configuración.

ADR-010: Microservicios orientados a eventos en Java 21 + Spring Boot (simulador incluido)

Tabla A35

ADR-010: ficha de la decisión

Campo	Valor
Estado	Aceptado
Fecha	2026-10-09
Decisores	Equipo de arquitectura del proyecto
Relacionados	ADR-002, ADR-003, ADR-004, ADR-005

Contexto

El objetivo general pide una solución basada en una **arquitectura distribuida**. El curso exige **Java** como lenguaje y permite Spring. El enunciado sugiere scripts de Python, JMeter o la línea de comandos de Kafka para simular eventos, y el stack de referencia incluye Elasticsearch y Airflow. Además, el grupo quiere **poder ejecutar el sistema en más de una máquina a la vez para repartir la carga**.

Hay que decidir:

1. cómo dividir el sistema en componentes;
2. con qué tecnología construirlos;
3. qué hacer con las herramientas sugeridas que no encajan.

Opciones consideradas

Tabla A36

ADR-010: opciones consideradas

Criterio	Monolito modular (1 app Spring) + simulador	Microservicios orientados a eventos (4 servicios)	Microservicios por tipo de sensor (≈ 8 servicios)
"Arquitectura distribuida" del enunciado	Débil	Sí	Sí
Repartir la carga entre máquinas	No (todo o nada)	Sí: cada servicio escala por separado	Sí

Criterio	Monolito modular (1 app Spring) + simulador	Microservicios orientados a eventos (4 servicios)	Microservicios por tipo de sensor (≈ 8 servicios)
Aislamiento de fallas	Bajo: una falla tumba todo	Alto	Alto
Complejidad operativa	Baja	Media (4 imágenes)	Alta (código duplicado)
Memoria	≈ 0,5 GB	≈ 1,6 GB	≈ 3 GB o más

Tabla A37

ADR-010: opciones consideradas

Simulador	Ventaja	Desventaja	Decisión
Scripts de Python	Rápido de escribir	Incumple "Java obligatorio"; duplica los contratos de eventos	Descartado
JMeter / Artillery	Bueno para estrés HTTP	No modela escenarios correlacionados (pánico → velocidad → fuga)	Opcional: el gateway REST de la app (`/api/v1/panic`) puede estresarse con JMeter si se desea
Simulador en Java/Spring Boot	Reutiliza los contratos de `common`; escenarios reproducibles; API de control para la demo	Más código	Elegido

Decisión

Cuatro microservicios Spring Boot 3.5 sobre Java 21, en un proyecto Maven de varios módulos, que se comunican **solo por Kafka** (estilo orientado a eventos):

Tabla A38

ADR-010: decisión

Servicio	Responsabilidad	Escala con
`sensor-simulator`	Emula los 5 tipos de sensores: tráfico de fondo, 10 escenarios (incluida la tabla 2.4), picos de carga, replay CSV con aleatorización, buffer en disco para eventos críticos y validación automática	Una instancia por nodo generador, cada una con un subconjunto de zonas
`ingestion-service`	Valida, normaliza, seudonimiza, deduplica y envía a la DLQ; gateway REST para la app móvil	Instancias del grupo `ingestion`
`correlation-engine`	Reglas por evidencia ponderada,	Instancias del grupo `correlation`

Servicio	Responsabilidad	Escala con
	incidentes, falsos positivos, seguimiento de placas	(estado en Redis)
`alert-service`	Persistencia en TimescaleDB, API REST, consola del operador en vivo (SSE), seguridad, tareas periódicas	Instancias detrás de un balanceador; las tareas usan ShedLock
`common` (librería)	Contratos de eventos con los campos del enunciado, catálogo de zonas, seudonimización	—

Decisiones relacionadas con el stack de referencia:

- **Elasticsearch: se omite.** El histórico se busca en TimescaleDB (ADR-003).
- **Airflow: se reemplaza** por Spring Scheduler + ShedLock (ADR-004).
- **Se agrega Prometheus** para métricas técnicas (ADR-008).
- **Kafka, PostgreSQL (TimescaleDB), Redis y Grafana: se adoptan.**

Calidad del código:

- 95 pruebas unitarias; las del motor usan las reglas reales del YAML y las de resiliencia simulan cortes de luz, fallas de Kafka y muertes de proceso;
- configuración externa por variables de entorno;
- comentarios en español que explican el porqué de cada decisión.

Consecuencias

Positivas

- Cumple el requisito de Java en el **100 % del código**, incluido el simulador.
- **Distribución real probada:** el motor de correlación corrió en dos máquinas lógicas a la vez con 6 particiones cada una, y la validación dio 9 de 9 en modo multi-nodo.
- Cada servicio se puede desplegar, escalar y depurar por separado. Si la base de datos falla, la detección en tiempo real sigue funcionando.
- Spring Boot aporta salud (`/actuator/health`), métricas, seguridad y Kafka con poca configuración.

Negativas (aceptadas)

- Cuatro servicios Java suman unos 1,6 GB de RAM, frente a ≈ 0,5 GB de un monolito.

- Depurar un flujo que cruza servicios es más difícil; se mitiga con el `event_id` que viaja desde el sensor hasta la alerta.
- Hay que mantener los contratos de eventos compatibles entre servicios; se resuelve con el módulo `common`, compartido.

Impacto en el negocio

- **Costo:** Java y Spring son libres y hay mucho personal disponible en Guatemala, lo que reduce el costo de mantenimiento a largo plazo.
- **Riesgo:** una falla en un componente no detiene todo el sistema. Elegir tecnologías de uso amplio reduce la dependencia de personas específicas.
- **Tiempo:** el proyecto Maven compila y prueba todo con un comando (`mvn install`), y las imágenes Docker se construyen igual para todos los servicios.